

DOCUMENTAZIONE CASO DI STUDIO

INGEGNERIA DELLA CONOSCENZA

2024-2025

Titolo: Classification system di videogiochi mediante tecniche di apprendimento supervisionato e recommender system basato su clustering

Progetto realizzato da: Antonio Silvestre, 697697, a.silvestre2@studenti.uniba.it

Repository: <https://github.com/silvestreantonio/ICON24-25.git>

SOMMARIO:

INTRODUZIONE

LINGUAGGIO E AMBIENTI UTILIZZATI

MANUALE D'UTILIZZO

PREPROCESSING

SCELTA CLASSIFICATORI

SCELTA IPERPARAMETRI E
OTTIMIZZAZIONE TRAMITE GRID SEARCH

CROSS VALIDATION

CLUSTERING E RECOMMENDATION

SVILUPPI FUTURI

INTRODUZIONE

Il caso di studio presentato riguarda l'analisi e la classificazione di dati relativi ai videogiochi, con l'obiettivo di fornire un sistema avanzato per la predizione del genere di un videogioco in base a diverse caratteristiche. L'importanza di questo sistema risiede nella crescente complessità del mercato dei videogiochi, che richiede strumenti automatizzati per supportare sia gli sviluppatori che gli utenti finali.

Il dataset utilizzato è "**metacritic_game_info**" (disponibile sul sito Kaggle, condiviso dal portale [metacritic.com](https://www.kaggle.com/datasets/metacriticgameinfo/metacritic-game-info)), che contiene informazioni dettagliate su una vasta gamma di videogiochi, tra cui il titolo, l'anno di pubblicazione, il publisher, il genere, le caratteristiche principali, la piattaforma, il punteggio critico (*metascore*), il punteggio degli utenti (*user_avg*) e il numero di giocatori supportato. Questi dati rappresentano la base per lo sviluppo di modelli di machine learning in grado di classificare correttamente i videogiochi in categorie di genere specifiche. L'architettura del progetto si articola in diverse fasi: preprocessing dei dati, addestramento di modelli di classificazione, confronto delle performance tra diversi algoritmi e implementazione di un sistema di raccomandazione basato su clustering. In particolare, il sistema è stato progettato per gestire la complessità delle caratteristiche categoriche attraverso tecniche di mappatura e bilanciamento del dataset, garantendo risultati accurati e affidabili.

LINGUAGGIO E AMBIENTI UTILIZZATI

Il progetto è stato sviluppato utilizzando **Python** nella versione 3.12.2, un linguaggio di programmazione versatile e ampiamente adottato nel campo dell'analisi dei dati e del machine learning. Python è stato scelto per la sua ricca libreria di strumenti dedicati all'elaborazione di dataset complessi, alla modellazione predittiva e alla visualizzazione dei risultati. Tra le librerie principali utilizzate ci sono:

- **Pandas** per la manipolazione e l'analisi dei dati,
- **NumPy** per il calcolo numerico efficiente,
- **Scikit-learn** per l'implementazione di algoritmi di machine learning (come Decision Tree, Random Forest e Support Vector Machine)
- **Matplotlib/Seaborn** per la creazione di grafici e visualizzazioni chiare ed efficaci.

Per quanto riguarda l'ambiente di sviluppo, sono stati utilizzati **Jupyter Notebook** e l'IDE **Visual Studio Code**, che hanno permesso di organizzare il codice in modo modulare e di eseguire test iterativi durante lo sviluppo. Inoltre, per gestire le dipendenze e garantire la riproducibilità del progetto, è stata adottata una configurazione basata su **Anaconda**, una distribuzione Python che include molte delle librerie necessarie preinstallate. Questo approccio ha semplificato la gestione delle versioni delle librerie e ha permesso di lavorare in un ambiente isolato e stabile.

Infine, per la fase di preprocessing e clustering, sono state utilizzate librerie specializzate come:

- **Imbalanced-Learn** per il bilanciamento del dataset
- **KModes** per l'applicazione di tecniche di clustering su dati categorici.

L'integrazione di queste tecnologie ha permesso di affrontare le sfide legate alla natura eterogenea del dataset, garantendo risultati accurati e affidabili.

MANUALE D'UTILIZZO

Per utilizzare il sistema è necessario eseguire il file main.py.

Una volta eseguito il main, il programma chiederà all'utente di scegliere quale sistema utilizzare. Una volta premuto il tasto scelto, l'utente dovrà inserire i dati richiesti come nome del gioco, anno di uscita, ecc..

Una volta inseriti tutti i dati richiesti il sistema restituirà:

- Il genere del gioco inserito se si è utilizzato il sistema di classificazione
- 5 giochi simili al gioco inserito se si è utilizzato il sistema di raccomandazione

Esempio di funzionamento Classification System:

```
Benvenuto utente
Premere:
(1) per scegliere il sistema di classificazione
(2) per scegliere il sistema di raccomandazione
-> 1
Benvenuto nel sistema di classificazione

Inserisci il nome di un titolo che ti è piaciuto
-> FIFA 23

Inserisci l'anno di uscita
-> 2022

Inserisci il publisher
-> EA

Inserisci l'intero corrispondente a una caratteristica distintiva del gioco:
(1) Open-World || (2) Baseball || (3) Golf || (4) Skateboarding || (5) Basketball || (6) Soccer || (7) Football || (8) Snowboarding || (9) Tennis || (10) Fantasy
(11) Sci-Fi || (12) Modern || (13) Historic || (14) Linear || (15) Horror || (16) 3D || (17) 2D || (18) Music || (19) GT/Street
(20) Rally || (21) FormulaOne || (22) Motorcycle || (23) Car/Combat || (24) Combat || (25) Kart || (26) Flight || (27) Stealth || (28) Arcade || (29) Real-Time
(30) Beat-Em-Up || (31) Application || (32) Ice Hockey || (33) VirtualLife || (34) General || (35) Point-and-Click || (36) Snow/Water || (37) Tactics || (38) Tactical || (39) Turn-Based
-> 6
```

```
Inserisci l'intero corrispondente alla console di gioco:
(1) Nintendo64 || (2) PlayStation || (3) Dreamcast || (4) Wii || (5) GameCube || (6) PlayStation2 || (7) Xbox
(8) GameBoyAdvance || (9) PlayStation3 || (10) Xbox360 || (11) PSP || (12) DS || (13) 3DS || (14) PlayStationVita
(15) WiiU || (16) PlayStation4 || (17) XboxOne || (18) Switch || (19) PC || (20) PlayStation5 || (21) Xbox Serie X\S
-> 16

Inserisci il voto che daresti a questo gioco
-> 9

Il genere del videogioco inserito è 'Sports'.
```

Esempio di funzionamento Recommender System:

```
Benvenuto utente
Premere:
(1) per scegliere il sistema di classificazione
(2) per scegliere il sistema di raccomandazione
-> 2
Benvenuto nel sistema di raccomandazione
Inserisci il nome di un titolo che ti è piaciuto
-> FIFA 23
Inserisci l'anno di uscita
-> 2022
Inserisci il publisher
-> EA
Inserisci l'intero corrispondente al genere del gioco:
(1) Action || (2) Action Adventure || (3) Action RPG || (4) RPG || (5) JRPG
(6) Rhythm || (7) Sports || (8) FPS || (9) TPS || (10) Fighting
(11) Racing || (12) Platformer || (13) Puzzle || (14) Strategy || (15) Adventure
(16) Sandbox || (17) MMORPG || (18) General || (19) Simulation || (20) Arcade
-> 7
```

```
Inserisci l'intero corrispondente alla console di gioco:
(1) Nintendo64 || (2) PlayStation || (3) Dreamcast || (4) Wii || (5) GameCube || (6) PlayStation2 || (7) Xbox
(8) GameBoyAdvance || (9) PlayStation3 || (10) Xbox360 || (11) PSP || (12) DS || (13) 3DS || (14) PlayStationVita
(15) WiiU || (16) PlayStation4 || (17) XboxOne || (18) Switch || (19) PC || (20) PlayStation5 || (21) Xbox Serie X\S
-> 16
Inserisci il voto che daresti a questo gioco
-> 9
I videogiochi raccomandati in base alle tue preferenze sono:
SSX 3
NHL 2002
NCAA Football 2003
SSX Tricky
Madden NFL 2004
```

PREPROCESSING

Questa fase include alcune modifiche fatte a mano ed altre tramite funzioni messe a disposizione dalla libreria python Pandas.

Il dataset originale si presenta come una lista ordinata di data objects aventi come attributi: *position*, *title*, *year*, *publisher*, *genre*, *platform*, *metascore*, *avg_userscore*, *no_player*. Gli unici valori numerici sono: *position*, *year*, *metascore* e *avg_userscore*.

Esempio dataset originario:

```
,Title,Year,Publisher,Genre,Platform,Metascore,Avg_Userscore,No_Players
0,The Legend of Zelda: Ocarina of Time,1998,Nintendo,Action Adventure;Fantasy,Nintendo64,99,9.1,1 Player
1,Tony Hawk's Pro Skater 2,2000,NeversoftEntertainment,Sports;Alternative;Skateboarding,PlayStation,98,7.4,1-2
2,Grand Theft Auto IV,2008,RockstarNorth,Action Adventure;Modern;Modern;Open-World,PlayStation3,98,7.5,1 Player
3,SoulCalibur,1999,Namco,Action;Fighting;3D,Dreamcast,98,8.6,1-2
4,Grand Theft Auto IV,2008,RockstarNorth,Action Adventure;Modern;Modern;Open-World,Xbox360,98,7.9,1 Player
5,Super Mario Galaxy,2007,Nintendo,Action;Platformer;Platformer;3D;3D,Wii,97,9.0,No Online Multiplayer
6,Super Mario Galaxy 2,2010,NintendoEADTokyo,Action;Platformer;Platformer;3D;3D,Wii,97,9.1,No Online Multiplayer
7,Grand Theft Auto V,2014,RockstarNorth,Action Adventure;Modern;Open-World,XboxOne,97,7.8,Up to 30
8,Grand Theft Auto V,2013,RockstarNorth,Modern;Action Adventure;Modern;Open-World,PlayStation3,97,8.3,Up to 16
9,Grand Theft Auto V,2013,RockstarNorth,Modern;Action Adventure;Modern;Open-World,Xbox360,97,8.3,Up to 16
10,Tony Hawk's Pro Skater 2,2000,Treyarch,Sports;Alternative;Skateboarding,Dreamcast,97,6.2,not specified
11,The Legend of Zelda: Breath of the Wild,2017,Nintendo,Action Adventure;Open-World,Switch,97,8.5,No Online Multiplayer
12,Tony Hawk's Pro Skater 3,2001,NeversoftEntertainment,Sports;Alternative;Skateboarding,PlayStation2,97,7.4,1-2
13,Perfect Dark,2000,RareLtd.,Action;Shooter;First-Person;Sci-Fi,Nintendo64,97,8.9,not specified
14,Grand Theft Auto V,2014,RockstarNorth,Action Adventure;Modern;Open-World,PlayStation4,97,8.3,Up to 30
15,Metroid Prime,2002,RetroStudios,Action;Shooter;First-Person;Sci-Fi,GameCube,97,9.2,1 Player
16,Grand Theft Auto III,2001,DMADesign,Action Adventure;Modern,PlayStation2,97,8.5,1 Player
17,Super Mario Odyssey,2017,Nintendo,Action;Platformer;3D,Switch,97,8.9,No Online Multiplayer
18,Halo: Combat Evolved,2001,BungieSoftware,Action;Shooter;First-Person;Sci-Fi,Xbox,97,8.6,1-4
19,NFL 2K1,2000,VisualConcepts,Sports;Traditional;Football;Sim,Dreamcast,97,6.4,not specified
```

Per quanto riguarda la qualità del dataset, non sono presenti valori mancanti ma pecca di consistenza, visto che molti attributi, essendo stringhe nel dataset iniziale, contengono ridondanze, errori concettuali e di conseguenza sono stati ricondotti successivamente a valori nominali basati su standard ben definiti. Il focus principale della fase di data preparation è l'attributo 'genre', che è stato modificato poiché risultava ambiguo e confusionario. Come è possibile notare dall'esempio, nel dataset originario, titoli dello stesso publisher e con la stessa identica struttura videoludica avevano attributi differenti e ridondanti. Di conseguenza si è scelto di omologare tutti gli attributi 'genre' verso un unico standard chiaro: "https://it.wikipedia.org/wiki/Categoria:Tipi_di_videogiochi", affiancando un nuovo attributo 'characteristic' che contiene quella che è definibile una caratteristica distintiva presente all'interno del prodotto (ambientazione, modalità di gioco, etc.), precedentemente presente all'intero dell'attributo 'genre'.

Infine anche gli attributi 'publisher' e 'no_players' sono stati modificati, dato che anch'essi contenevano ridondanze ed errori, nello specifico per quanto riguarda il primo, i valori di ogni tupla di 'publisher' sono stati sostituiti, visto che erroneamente nel dataset sono stati inseriti gli studi di sviluppo e non i veri publisher dei prodotti, mentre per quanto riguarda il secondo, alcuni valori come già detto erano ridondanti e scritti in maniera diversi, di conseguenza ricondotti a una forma unica.

Il dataset iniziale conteneva 5000 data objects, di conseguenza è stato necessario ridurre la dimensione del dataset a 488 data objects, visto che l'obiettivo di questo caso di studio era di creare un classification system e recommender system che classificasse e consigliasse i prodotti più apprezzati di tutti i tempi, scegliendo così data objects con i valori dell'attributo 'metascore' >= 90, ottenendo così solo le eccellenze del settore.

Esempio dataset modificato:

```
id;position;title;year;publisher;genre;characteristic;platform;metascore;user_avg;no_players
1;1;The Legend of Zelda: Ocarina of Time;1998;Nintendo;Action Adventure;Fantasy;Nintendo64;99;9.1;1 Player
2;2;Tony Hawk's Pro Skater 2;2000;Activision;Sports;Skateboarding;PlayStation;98;7.4;not specified
3;3;Grand Theft Auto IV;2008;RockstarGames;Action Adventure;Open-World;PlayStation3;98;7.5;1 Player
4;4;SoulCalibur;1999;Bandai;Fighting;3D;Dreamcast;98;8.6;1 to 2
5;5;Grand Theft Auto IV;2008;RockstarGames;Action Adventure;Open-World;Xbox360;98;7.9;1 Player
6;6;Super Mario Galaxy;2007;Nintendo;Platformer;3D;Wii;97;9.0;No Online Multiplayer
7;7;Super Mario Galaxy 2;2010;Nintendo;Platformer;3D;Wii;97;9.1;No Online Multiplayer
8;8;Grand Theft Auto V;2014;RockstarGames;Action Adventure;Open-World;XboxOne;97;7.8;1 to 30
9;9;Grand Theft Auto V;2013;RockstarGames;Action Adventure;Open-World;PlayStation3;97;8.3;1 to 16
10;10;Grand Theft Auto V;2013;RockstarGames;Action Adventure;Open-World;Xbox360;97;8.3;1 to 16
11;11;Tony Hawk's Pro Skater 2;2000;Activision;Sports;Skateboarding;Dreamcast;97;6.2;not specified
12;12;The Legend of Zelda: Breath of the Wild;2017;Nintendo;Action Adventure;Open-World;Switch;97;8.5;No Online Multiplayer
```

Le modifiche fatte tramite la libreria pandas sono le seguenti:

- cancellazione delle colonne “*id*” e “*position*”
- discretizzazione della colonna “*year*” definendo range ben definiti
- associazione di ogni attributo categorico ad un numerico mediante Label Encoder e ulteriore riconduzione di essi in categorico, poiché sia la SVM sia il Decision Tree messi a disposizione da scikit-learn non funzionano su attributi categorici.

Inoltre sono state fatte ulteriori operazioni sul dataset, ma quest’ultime sono riguardanti il sistema che si decide di utilizzare.

Esempio dataset dopo l’esecuzione di Preprocess.py:

```
title,year_range,publisher,genre,characteristic,platform,no_players,metascore,user_avg
The Legend of Zelda: Ocarina of Time,1995–2000,Nintendo,Action Adventure,Fantasy,Nintendo64,1 Player,99,9.1
Tony Hawk's Pro Skater 2,1995–2000,Activision,Sports,Skateboarding,PlayStation,not specified,98,7.4
Grand Theft Auto IV,2005–2010,RockstarGames,Action Adventure,Open-World,PlayStation3,1 Player,98,7.5
SoulCalibur,1995–2000,Bandai,Fighting,3D,Dreamcast,1 to 2,98,8.6
Grand Theft Auto IV,2005–2010,RockstarGames,Action Adventure,Open-World,Xbox360,1 Player,98,7.9
Super Mario Galaxy,2005–2010,Nintendo,Platformer,3D,Wii,No Online Multiplayer,97,9.0
Super Mario Galaxy 2,2005–2010,Nintendo,Platformer,3D,Wii,No Online Multiplayer,97,9.1
Grand Theft Auto V,2010–2015,RockstarGames,Action Adventure,Open-World,XboxOne,1 to 30,97,7.8
Grand Theft Auto V,2010–2015,RockstarGames,Action Adventure,Open-World,PlayStation3,1 to 16,97,8.3
Grand Theft Auto V,2010–2015,RockstarGames,Action Adventure,Open-World,Xbox360,1 to 16,97,8.3
Tony Hawk's Pro Skater 2,1995–2000,Activision,Sports,Skateboarding,Dreamcast,not specified,97,6.2
The Legend of Zelda: Breath of the Wild,2015–2020,Nintendo,Action Adventure,Open-World,Switch,No Online Multiplayer,97,8.5
```

SCELTA CLASSIFICATORI

Per creare il classification system in grado di classificare un prodotto dato in input dall'utente, ho utilizzato tecniche di apprendimento supervisionato e confrontandole tra loro attraverso valutazioni espresse in base a precise metriche, ho scelto il classificatore più performante tra di essi.

Il processo di classificazione è suddiviso in tre fasi:

- training, si produce un modello da un insieme di dati detto training set
- stima dell'accuratezza, mediante metriche utili a stimare le performance su un insieme di dati diverso dal training set, inizialmente validation set mediante cross validation e poi test set classificando istanze di classe ignota
- scelta del classificatore più adatto per il sistema

La classificazione nel caso di studio è stata utilizzata con lo scopo di predire, tramite addestramento sul dataset precedentemente ottenuto, il genere di un videogioco fornito dall'utente. La feature target, ossia la label da predire è la colonna '*genre*'. Per procedere con la classificazione, ovviamente bisogna scegliere il classificatore più adatto, ossia quello che performi ottimamente sia un training set e che sappia generalizzare su un test set. Nel machine learning esiste il teorema "No free Lunch" che afferma che non esiste un algoritmo che vada bene per qualsiasi problema e, di conseguenza, sono necessarie varie prove per trovare il modello di predizione più accurato, valutando le performance di ciascuna alternativa, al fine di trovare la più adatta.

Il metodo utilizzato è stato quello di considerare i classificatori le cui caratteristiche sembravano aderenti al caso di studio, per poi valutarne le performance utilizzando la cross validation, evitando che i classificatori scelti vadano in overfitting, ovvero, buonissime performance sul training set ma basse performance sul test set, successivamente effettuare un confronto tra i classificatori presi in esame. Una volta trovato il più performante, quest'ultimo viene stato utilizzato per le predizioni del sistema.

La scelta degli algoritmi da testare si è basata sulla mia curiosità di confrontare modelli molto diversi tra loro, inizialmente solo SVM e Random Forest a cui successivamente è stato aggiunto il Decision Tree.

Gli algoritmi più nello specifico sono i seguenti:

1. C-Support Vector Classification (SVC):

L'algoritmo SVM seppur utilizzato per classificazioni binarie, può essere impiegato per problemi di classificazione multi-classe utilizzando la metodologia one-vs-one oppure la one-vs-rest. Nello specifico nella prima, si creano $k(k-1)/2$ classificatori, dove k è il numero di classi, che effettuano classificazione su coppie di classi, per poi assegnare come classe finale quella con più assegnazioni, mentre nella seconda si creano k classificatori quanto i k feature target e si fa un confronto tra quella selezionata e il resto scegliendo la majority class. La SVM si basa sull'idea di trovare un iperpiano che divida il set di dati in due o più classi su x dimensioni spaziali dove x è il numero di classi. I punti dati più vicini all'iperpiano sono detti vettori di supporto e rappresentano le possibili classi di appartenenza. Un iperpiano linearmente separabile è un iperpiano in cui è semplice distinguere due classi, il problema è trovare quale tra le infinite rette che rappresentano l'iperpiano risulti ottimale, ossia quella che generi il minimo errore di classificazione su una nuova osservazione. Il metodo del kernel consente di modellare modelli non lineari di dimensioni superiori. Il suo scopo è di prendere i dati come input e trasformarli nella forma richiesta qualora non sia possibile determinare un iperpiano linearmente separabile. In questo caso useremo il metodo kernel con l'SVM poiché il modello in questione è non lineare.

I principali vantaggi di questo algoritmo sono i seguenti:

- Efficace in dimensioni spaziali elevate
- Efficienza della memoria, poiché solo un sottoinsieme dei punti di allenamento viene utilizzato nel processo decisionale effettivo di assegnazione
- Versatilità, grazie alla capacità di applicare nuovi kernel portando a una maggiore performance di classificazione.

I suoi principali svantaggi sono:

- Interpretazione non semplice
- Metodo non probabilistico, poiché il classificatore funziona posizionando gli oggetti sopra e sotto un iperpiano di classificazione, non esiste un'interpretazione probabilistica diretta per l'appartenenza al gruppo
- Richiede adattamento per tutte quelle variabile non numeriche

2. Random Forest Classifier:

La Random Forest è un bagging di alberi di decisione, i quali vengono uniti per ottenere una previsione più accurata e stabile. Ogni albero in una random forest impara da un campione casuale di dati. I campioni vengono disegnati con la sostituzione, nota come bootstrap, ovvero, alcuni campioni verranno utilizzati più volte in un singolo albero. L'idea è che addestrando ciascun albero su campioni diversi, sebbene ogni albero possa presentare una varianza elevata rispetto a una particolare serie di dati di addestramento, nel complesso la foresta avrà una varianza inferiore, in modo da avere le predizioni finali vicine al risultato.

I vantaggi del Random Forest sono nella versatilità, essendo utilizzabile sia per problemi di regressione che di classificazione, funzionando bene con una combinazione di caratteristiche numeriche e categoriche.

Un altro vantaggio sta nell'avere gli iperparametri predefiniti ottimali, perché producono un buon risultato di previsione, consentendo anche un notevole miglioramento di essi, e di conseguenza della previsione. In generale, questi algoritmi sono veloci da addestrare, ma piuttosto lenti nel creare previsioni una volta che sono stati addestrati, poiché richiedono un alto numero di alberi per ottenere risultati accurati.

3. Decision Tree:

L'albero di decisione è metodo non parametrico, utilizzabile sia per task di classificazione sia per regressione. L'algoritmo messo a disposizione da scikit-learn si basa una versione ottimizzata di CART, che similmente a C4.5 costruisce alberi in partizioni binarie usando come feature e threshold quella che porta il più grande valore di information gain per ogni nodo. Tuttavia, benché gli alberi di decisioni siano nati per essere utilizzati su attributi categorici, la versione messa a disposizione da scikit-learn non supporta quest'ultimi e perciò è stato necessario attuare nella fase di preprocessing una conversione di tutti gli attributi categorici a numerici mediante Label Encoder.

I vantaggi del Decision Tree sono:

- Facilità di interpretazione e visualizzazione
- Richiede poca data preparation
- Performante sia su dati categorici e numerici e particolarmente adatto per problemi multy-way

I principali svantaggi sono:

- Alberi troppo complessi potrebbero portare ad overfitting
- Piccole variazioni sui dati possono generare alberi altamente differenti.

SCelta IPERPARAMETRI E OTTIMIZZAZIONE TRAMITE GRID SEARCH

Alcuni dei classificatori utilizzati richiedono un'attenta configurazione degli iperparametri, poiché la loro scelta influisce direttamente sulle prestazioni del modello. Gli iperparametri sono parametri che non vengono appresi durante l'addestramento, ma devono essere forniti in input prima dell'apprendimento. La combinazione ottimale di iperparametri è fondamentale per ottenere il miglior risultato possibile.

Per identificare la miglior combinazione di iperparametri, è stata utilizzata la tecnica della Grid Search, che esplora sistematicamente diverse configurazioni di iperparametri attraverso una ricerca esaustiva. Per ogni classificatore, è stata effettuata una ricerca nello spazio degli iperparametri al fine di massimizzare il punteggio di cross-validation.

1. Random Forest

Per il modello Random Forest, sono stati testati i seguenti iperparametri:

- `n_estimators`: numero di alberi nella foresta, con valori [10, 100, 1000]
- `max_features`: numero di feature da considerare per determinare il miglior split, con valori ['sqrt', 'log2']

2. Support Vector Classifier (SVC)

Per il modello SVC , sono stati testati i seguenti iperparametri:

- C : parametro di regolarizzazione, con valori [50, 10, 1.0]
- kernel: tipo di kernel utilizzato, con valori ['poly', 'rbf']
- gamma : coefficiente per il kernel, con valore ['auto']

3. Decision Tree

Il modello Decision Tree è un algoritmo non parametrico che non richiede la configurazione di iperparametri specifici. Pertanto, non è stata necessaria alcuna ricerca di iperparametri per questo classificatore.

Grid Search applicata a SVC e Random Forest

```
# Definizione delle griglie di iperparametri
svc_param_grid = {
    'C': [50, 10, 1.0], # Parametro regolarizzatore
    'kernel': ['poly', 'rbf'], # Tipo di kernel scelto
    'gamma': ['auto'] # Coefficiente per il kernel
}
rf_param_grid = {
    'n_estimators': [10, 100, 1000], # Numeri di alberi nella foresta
    'max_features': ['sqrt', 'log2'], # Numero di feature da considerare per il miglior split
}

# Ricerca degli iperparametri ottimali con GridSearch per SVC
print("\nRicerca degli iperparametri per SVC...")
svc_grid = GridSearchCV(
    SVC(probability=True),
    param_grid=svc_param_grid,
    scoring='accuracy',
    cv=3, # Numero di fold per la cross-validation
    n_jobs=-1 # Usa tutti i core disponibili
)
svc_grid.fit(xtrain, ytrain)
print("\n\nMigliori iperparametri per SVC:\n", svc_grid.best_params_)

# Ricerca degli iperparametri ottimali con GridSearch per Random Forest
print("\nRicerca degli iperparametri per Random Forest...")
rf_grid = GridSearchCV(
    RandomForestClassifier(random_state=42),
    param_grid=rf_param_grid,
    scoring='accuracy',
    cv=3, # Numero di fold per la cross-validation
    n_jobs=-1 # Usa tutti i core disponibili
)
rf_grid.fit(Xtrain, ytrain)
print("\n\nMigliori iperparametri per Random Forest:\n", rf_grid.best_params_)
```

I migliori iperparametri trovati dalla *Grid Search* sono:

```
Migliori iperparametri per SVC:
{'C': 1.0, 'gamma': 'auto', 'kernel': 'rbf'}
```

```
Migliori iperparametri per Random Forest:
{'max_features': 'log2', 'n_estimators': 100}
```

Precision: $TP / (TP + FP)$ rapporto tra le istanze ritrovate e definite come rilevanti e le istanze ritrovate;

Recall: $TP / (TP + FN)$ rapporto tra le istanze ritrovate e definite come rilevanti e le istanze effettivamente rilevanti;

Accuratezza: $(TP + TN) / (TP + FP + TN + FN)$, è il rapporto tra le istanze correttamente predette e il totale delle istanze presenti. Testa l'abilità del modello nel predire correttamente le due classi.

CROSS VALIDATION

Cross-Validation e Valutazione dei Modelli

Il processo di cross-validation è una tecnica fondamentale per valutare le prestazioni di modelli di machine learning in modo robusto e affidabile. Nel codice fornito, viene implementata una doppia k-fold cross-validation, che combina una validazione incrociata esterna (outer k-fold) con una interna (inner k-fold). Questo approccio garantisce una stima accurata delle prestazioni dei modelli, riducendo il rischio di overfitting e fornendo una visione più realistica del loro comportamento su dati non visti.

Outer K-Fold

Il dataset viene suddiviso in 5 fold esterni ($outer_kfold = KFold(n_splits=5)$). Per ciascun fold esterno:

- Un fold viene utilizzato come test set
- I restanti fold vengono utilizzati come training set

```
outer_kfold = KFold(n_splits=5, shuffle=True, random_state=42) # Outer K-Fold
for fold, (train_idx, test_idx) in enumerate(outer_kfold.split(X, y), start=1):
    print(f"\nOuter Fold {fold}:")
    X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
    y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]
```

X_train e y_train rappresentano i dati di training per il fold corrente, mentre X_test e y_test sono i dati di test.

Inner K-Fold

All'interno di ciascun fold esterno, il training set viene ulteriormente suddiviso in 5 fold interni ($inner_kfold = KFold(n_splits=5)$) per eseguire una validazione incrociata interna durante l'addestramento dei modelli. Questo processo valuta le prestazioni del modello su dati non visti durante l'addestramento interno.

```
print(f" Performing inner cross-validation for models...")
svc_inner_results = cross_validate(svc_model, X_train, y_train, cv=inner_kfold, scoring=scoring)
```

Qui, la funzione **cross_validate** esegue la validazione incrociata interna per il modello SVC, calcolando metriche come accuracy, precision e recall.

Addestramento Finale e Valutazione

Dopo la validazione incrociata interna, il modello viene addestrato sul training set completo del fold esterno e valutato sul test set esterno:

```
svc_model.fit(X_train, y_train)
svc_preds = svc_model.predict(X_test)
svc_combined_metrics['accuracy'].append(accuracy_score(y_test, svc_preds))
svc_combined_metrics['precision'].append(precision_score(y_test, svc_preds, average='macro', zero_division=0))
svc_combined_metrics['recall'].append(recall_score(y_test, svc_preds, average='macro', zero_division=0))
```

Questo processo viene ripetuto per tutti i modelli (SVC, Random Forest, Decision Tree).

Matrici di Confusione

Le matrici di confusione sono uno strumento essenziale per valutare le prestazioni di un modello di classificazione. Una matrice di confusione mostra quanti esempi sono stati classificati correttamente e quali errori sono stati commessi.

Nel codice, la funzione **plot_confusion_matrix** genera una matrice di confusione per ciascun modello:

```
# Funzione per stampare la matrice di confusione
def plot_confusion_matrix(y_true, y_pred, model_name, genre_mapping):
    labels = [genre for genre, idx in sorted(genre_mapping.items(), key=lambda x: x[1])]
    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(8, 6))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=labels, yticklabels=labels)
    plt.title(f'Matrice di Confusione - {model_name}')
    plt.ylabel('Reale')
    plt.xlabel('Predetto')
    plt.xticks(rotation=45)
    plt.yticks(rotation=0)
    plt.tight_layout()
    plt.show()
```

Questa funzione viene chiamata durante la valutazione dettagliata dei modelli:

```
svc_metrics = train_and_evaluate_model(svc_model, prepInfo.x_train, prepInfo.y_train, prepInfo.x_test, prepInfo.y_test, "SVC", prepInfo.genreD)
rf_metrics = train_and_evaluate_model(rand_model, prepInfo.x_train, prepInfo.y_train, prepInfo.x_test, prepInfo.y_test, "Random Forest", prepInfo.genreD)
tree_metrics = train_and_evaluate_model(tree_model, prepInfo.x_train, prepInfo.y_train, prepInfo.x_test, prepInfo.y_test, "Decision Tree", prepInfo.genreD)
```

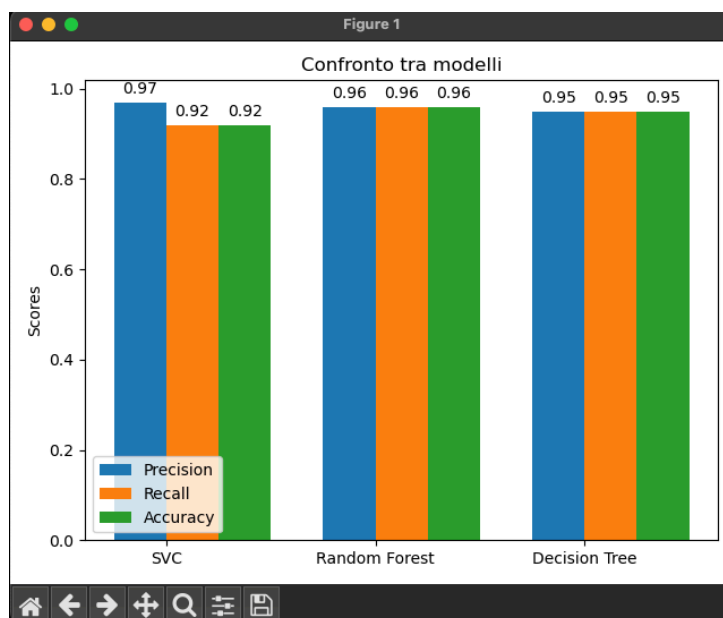

Risultati Finali

Al termine della doppia cross-validation, viene creata una tabella (models_scores_table) che mostra le metriche medie combinate per ciascun modello:

```
models_scores_table = pd.DataFrame({
    'SVC': [
        np.mean(svc_combined_metrics['accuracy']),
        np.mean(svc_combined_metrics['precision']),
        np.mean(svc_combined_metrics['recall']),
    ],
    'Random Forest': [
        np.mean(rfc_combined_metrics['accuracy']),
        np.mean(rfc_combined_metrics['precision']),
        np.mean(rfc_combined_metrics['recall']),
    ],
    'Decision Tree': [
        np.mean(tree_combined_metrics['accuracy']),
        np.mean(tree_combined_metrics['precision']),
        np.mean(tree_combined_metrics['recall']),
    ],
}, index=['Accuracy', 'Precision', 'Recall'])
```

L'output finale mostra che il modello Random Forest emerge come il migliore in termini di Accuracy e Recall, mentre il modello SVC ottiene la precisione più alta:

	SVC	Random Forest	Decision Tree	Best Score
Accuracy	0.916107	0.955893	0.953248	Random Forest
Precision	0.966872	0.956831	0.954475	SVC
Recall	0.916941	0.956564	0.954755	Random Forest



Sulla base dei risultati emersi dalla cross-validation, ho scelto il modello Random Forest per questo compito. La sua superiorità in termini di accuratezza e recall lo rende particolarmente adatto per applicazioni in cui è fondamentale minimizzare gli errori di classificazione.

CLUSTERING E RECOMMENDATION

Il **clustering** è una metodologia di apprendimento non supervisionato che consente di identificare e raggruppare elementi simili all'interno di un dataset, creando **cluster**, ovvero gruppi di elementi che risultano conformi a un elemento medio, detto **medoide** nel caso di dati categorici. Questa tecnica è particolarmente utile quando si desidera scoprire pattern nascosti o correlazioni tra i dati senza avere etichette predefinite.

Nel contesto del caso di studio, il clustering è stato adottato con l'obiettivo di individuare similarità e correlazioni tra i dati che non dipendessero unicamente dall'attributo "genere". Questo approccio è stato scelto per arricchire le informazioni disponibili e sfruttarle successivamente nella costruzione di un sistema di recommendation. Infatti, basarsi esclusivamente sul genere di un videogioco potrebbe limitare la capacità di catturare relazioni più complesse e sfumate tra gli elementi del dataset.

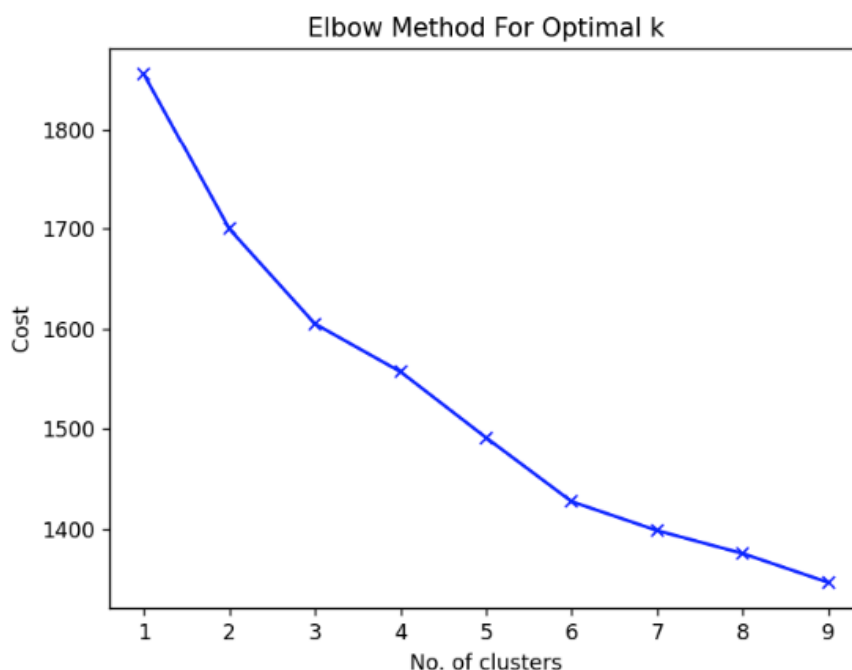
Determinazione del Numero Ottimale di Cluster

Un aspetto critico nell'applicazione del clustering è la determinazione del numero ottimale di cluster. Per affrontare questa sfida, ho utilizzato il metodo empirico noto come **"Elbow Method"** (metodo del gomito). Questo metodo consiste nel tracciare la variazione della funzione di costo (o **inerzia**) in funzione del numero di cluster e individuare il punto in cui l'aggiunta di ulteriori cluster porta a miglioramenti marginali.

Di seguito è riportato un esempio di codice per applicare il metodo Elbow:

```
# metodo del gomito
def elbowMethod(dataset):
    cost = []
    K = range(1, 10)
    for num_clusters in list(K):
        km = KMeans(n_clusters=num_clusters, init="random", n_init=5, verbose=1)
        km.fit_predict(dataset)
        cost.append(km.cost_)

    plt.plot(K, cost, 'bx-')
    plt.xlabel('No. of clusters')
    plt.ylabel('Cost')
    plt.title('Elbow Method For Optimal k')
    plt.show()
```



L'analisi del grafico generato dal metodo Elbow ha evidenziato che il "gomito" si verifica intorno a **6 cluster**. Questo numero rappresenta un buon compromesso tra:

- La capacità di distinguere gruppi significativi di giochi con caratteristiche simili.
- La semplicità interpretativa e la praticità di utilizzo dei cluster per il sistema di recommendation.

Il grafico mostra chiaramente che, dopo 6 cluster, la riduzione del costo diventa marginale, suggerendo che ulteriori suddivisioni non apporterebbero benefici significativi. Inoltre, la natura del dataset e la sua struttura intrinseca hanno supportato questa scelta. Ad esempio:

- Il dataset contiene attributi categorici come *genre*, *platform*, *publisher*, ecc., che riflettono diverse combinazioni di caratteristiche dei videogiochi.
- 6 cluster sembrano essere sufficienti per catturare le principali tipologie di giochi e preferenze degli utenti, senza sovra-adattare il modello o rendere i cluster troppo specifici.

Infine, durante lo sviluppo del progetto, sono state testate diverse configurazioni (es. 4, 5, 6, 7 cluster), e **6 cluster** è risultato essere il numero che produceva i risultati migliori in termini di:

- Qualità delle raccomandazioni generate.
- Coerenza interna dei cluster (es. giochi all'interno dello stesso cluster erano effettivamente simili).
- Performance computazionale (più cluster richiedono più tempo di elaborazione).

Applicazione di K-Modes

Una volta determinato il numero ottimale di cluster, ho applicato l'algoritmo **K-Modes** al dataset preprocessato. L'algoritmo K-Modes è una variante di K-Means specificamente progettata per gestire dati categorici. A differenza di K-Means, K-Modes utilizza una misura di similarità basata su attributi categorici, sostituendo il calcolo della media con quello della **moda**. In particolare, K-Modes minimizza una funzione di costo basata su un metodo **frequency-based**, che calcola la frequenza delle modalità comuni tra gli elementi all'interno di ciascun cluster.

Di seguito è riportato un esempio di codice Python per applicare K-Modes:

```
kmode = KModes(n_clusters=6, init="random", n_init=5, verbose=1)
dataset['cluster'] = kmode.fit_predict(dataset)
```

Calcolo delle Similarità

Dopo aver applicato il clustering, ho calcolato la similarità tra l'input dell'utente (*userInputGame*) e i medoidi di ciascun cluster. Questo è stato fatto utilizzando la libreria *fuzzywuzzy*, che calcola il grado di similarità tra stringhe basandosi su una metrica di distanza fuzzy. Ad esempio, per ogni medoide, ho sommato i punteggi di similarità per gli attributi *genre*, *title*, *year_range*, *publisher*, *characteristic*, *platform* e *user_avg_range*.

Di seguito è riportato un esempio di codice per calcolare le similarità:

```
# calcolo similarità con gli elementi del cluster
def similaritiesCluster(cluster, userInputGame):
    cluster['sum'] = 0
    for i in range(0, len(cluster)):
        rowSum = fuzz.ratio(cluster['genre'].values[i], userInputGame['genre'].values[0])
        rowSum = rowSum + fuzz.ratio(cluster['title'].values[i], userInputGame['title'].values[0])
        rowSum = rowSum + fuzz.ratio(cluster['year_range'].values[i], userInputGame['year_range'].values[0])
        rowSum = rowSum + fuzz.ratio(cluster['publisher'].values[i], userInputGame['publisher'].values[0])
        rowSum = rowSum + fuzz.ratio(cluster['characteristic'].values[i], userInputGame['characteristic'].values[0])
        rowSum = rowSum + fuzz.ratio(cluster['platform'].values[i], userInputGame['platform'].values[0])
        rowSum = rowSum + fuzz.ratio(cluster['user_avg_range'].values[i], userInputGame['user_avg'].values[0])
    cluster['sum'].values[i] = rowSum
```

Selezione del Cluster e Raccomandazione

Dopo aver calcolato le similarità, ho selezionato il cluster corrispondente al medoide con il punteggio più alto. Successivamente, ho calcolato la similarità tra l'input dell'utente e tutti gli elementi del cluster selezionato, ordinando i risultati in base alla similarità totale. I primi 5 elementi sono stati quindi raccomandati all'utente.

```
def clusterSelection(userInputGame, new_data_medoids, cluster1, cluster2, cluster3, cluster4, cluster5, cluster6):  
    if new_data_medoids['cluster'].values[0] == 0:  
        cluster_1 = pd.DataFrame(cluster1, columns=['title', 'year_range', 'publisher', 'genre', 'characteristic', 'platform', 'user_avg_range'])  
        cluster_1.to_csv('./datasets/cluster1.csv', index=False)  
        cluster_1 = pd.read_csv('./datasets/cluster1.csv', sep=',')  
        similaritesCluster(cluster_1, userInputGame)  
        cluster_1.sort_values(by=['sum'], ascending=False, inplace=True)  
        print("I videogiochi raccomandati in base alle tue preferenze sono:")  
        for i in range(0, 5):  
            print(cluster_1['title'].values[i])
```

Vantaggi e Limitazioni

L'adozione del clustering ha portato numerosi vantaggi al nostro progetto:

- Ha permesso di scoprire relazioni nascoste tra gli attributi del dataset, andando oltre la semplice classificazione per genere.
- Ha fornito una base solida per la costruzione di un sistema di recommendation più accurato e personalizzato.

È importante, però, sottolineare alcune limitazioni:

- La qualità dei cluster dipende fortemente dalla scelta degli attributi utilizzati. Attributi irrilevanti o ridondanti possono compromettere i risultati.
- L'algoritmo K-Modes, pur essendo adatto per dati categorici, può essere sensibile alla presenza di outlier o a distribuzioni non uniformi dei dati.

SVILUPPI FUTURI

1. Interfaccia Utente

- Sviluppare un'interfaccia utente grafica per rendere il modello accessibile a utenti non tecnici:
 - Un'applicazione web dove gli utenti possono inserire i dettagli di un videogioco e ottenere la predizione del genere.
 - Integrazione con piattaforme di gaming o database pubblici per automatizzare l'inserimento dei dati.

2. Predizione Multi-Etichetta

- Attualmente, il modello predice un singolo genere per ogni videogioco. Tuttavia, molti giochi appartengono a più generi contemporaneamente (ad esempio, "Action Adventure" e "Open-World"). Si potrebbe implementare un approccio di classificazione multi-etichetta per prevedere più generi contemporaneamente.

3. Analisi Temporale

- Esplorare l'evoluzione dei generi nel tempo:
 - Analizzare come i generi sono cambiati tra diverse epoche (year_range).
 - Identificare tendenze storiche nei generi di successo o nelle caratteristiche dei giochi.